

```
!pip install google-genai youtube-transcript-api
!pip install --upgrade youtube-transcript-api
```

```
Requirement already satisfied: google-genai in /usr/local/lib/python3.12/dist-packages (2.10.0)
Collecting youtube-transcript-api
  Downloading youtube_transcript_api-1.2.4-py3-none-any.whl.metadata (24 kB)
Requirement already satisfied: anyio<5.0.0,>=4.8.0 in /usr/local/lib/python3.12/dist-packages (from google-genai) (4.14.0)
Requirement already satisfied: google-auth<3.0.0,>=2.48.1 in /usr/local/lib/python3.12/dist-packages (from google-auth[requests]>=2.48.1->google-genai) (2.48.1)
Requirement already satisfied: httpx<1.0.0,>=0.28.1 in /usr/local/lib/python3.12/dist-packages (from google-genai) (0.28.1)
Requirement already satisfied: pydantic<3.0.0,>=2.12.5 in /usr/local/lib/python3.12/dist-packages (from google-genai) (2.13.1)
Requirement already satisfied: requests<3.0.0,>=2.28.1 in /usr/local/lib/python3.12/dist-packages (from google-genai) (2.32.4)
Requirement already satisfied: tenacity<9.2.0,>=8.2.3 in /usr/local/lib/python3.12/dist-packages (from google-genai) (9.1.4)
Requirement already satisfied: websockets<17.0,>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from google-genai) (15.0.7)
Requirement already satisfied: typing-extensions<5.0.0,>=4.14.0 in /usr/local/lib/python3.12/dist-packages (from google-genai) (4.14.0)
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from google-genai) (1.9.0)
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from google-genai) (1.3.1)
Requirement already satisfied: defusedxml<0.8.0,>=0.7.1 in /usr/local/lib/python3.12/dist-packages (from youtube-transcript-api) (0.7.1)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5.0.0,>=4.8.0->google-genai) (3.10.1)
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from google-auth<3.0.0,>=2.48.1->google-genai) (0.4.1)
Requirement already satisfied: cryptography>=38.0.3 in /usr/local/lib/python3.12/dist-packages (from google-auth<3.0.0,>=2.48.1->google-genai) (45.0.7)
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-packages (from google-auth<3.0.0,>=2.48.1->google-genai) (4.9.1)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0,>=0.28.1->google-genai) (2025.7.14)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0,>=0.28.1->google-genai) (1.0.7)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1.0.0,>=0.28.1->google-genai) (0.14.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.12.5->google-genai) (0.7.0)
Requirement already satisfied: pydantic-core==2.46.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.12.5->google-genai) (2.27.1)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.12.5->google-genai) (0.10.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.28.1->google-genai) (3.4.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.28.1->google-genai) (2.3.0)
Requirement already satisfied: cffi>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from cryptography>=38.0.3->google-genai) (2.0.0)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.12/dist-packages (from pyasn1-modules>=0.2.1->google-genai) (0.6.1)
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cffi>=2.0.0->cryptography>=38.0.3->google-genai) (2.23)
Downloading youtube_transcript_api-1.2.4-py3-none-any.whl (485 kB)
485.2/485.2 kB 15.3 MB/s eta 0:00:00
Installing collected packages: youtube-transcript-api
Successfully installed youtube-transcript-api-1.2.4
Requirement already satisfied: youtube-transcript-api in /usr/local/lib/python3.12/dist-packages (1.2.4)
Requirement already satisfied: defusedxml<0.8.0,>=0.7.1 in /usr/local/lib/python3.12/dist-packages (from youtube-transcript-api) (0.7.1)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from youtube-transcript-api) (2.32.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->youtube-transcript-api) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->youtube-transcript-api) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->youtube-transcript-api) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->youtube-transcript-api) (2025.7.14)
```

```
from google.colab import userdata
GOOGLE_API_KEY = userdata.get('GOOGLE_API_KEY')
```

```
from youtube_transcript_api import YouTubeTranscriptApi

def get_transcript(video_id, languages=['en']):
    transcript = YouTubeTranscriptApi().fetch(video_id, languages=languages)
    print(transcript)

    text = " ".join([item.text for item in transcript])
    return text

# Correct usage
video_id = "nLj-DGIC_xM"
print(get_transcript(video_id))
```

```

    FetchedTranscript(snippets=[
        FetchedTranscriptSnippet(
            text='Hello everyone, my name is Priya and I',
            start=0.8,
            duration=4.88
        ),
        FetchedTranscriptSnippet(
            text='Hello everyone, my name is Priya and I welcome all of you to the session. In this session, we are going to discuss about data science and its applications in the field of artificial intelligence.',
            start=5.28,
            duration=10.0
        )
    ])

```

```
from google import genai

client = genai.Client(api_key=GOOGLE_API_KEY)
```

```
import json
import re

def analyze_video(transcript):
    prompt = f"""
    Analyze this YouTube video transcript and return a structured breakdown.

    Transcript:
    {transcript}

```



```
Return in JSON format with these exact keys:
- summary (2-3 sentence short summary)
- detailed_explanation (a longer paragraph explaining the video's content)
- key_points (list of short bullet points)
- insights (list of short bullet points, important takeaways)
- qna (list of objects, each with "question" and "answer" keys – exactly 5 items)
- topics (list of up to 6 short topic/keyword tags for this video)
```

Rules:

- Output ONLY JSON
- No extra text, no markdown code fences

```
"""
    response = client.models.generate_content(
        model="gemini-3.5-flash",
        contents=prompt
    )
    return response.text
```

```
video_id = "nLj-DGIC_xM" # replace
transcript = get_transcript(video_id)
```

```
result = analyze_video(transcript)
print(result)
```

```
FetchedTranscript(snippets=[FetchedTranscriptSnippet(text='Hello everyone, my name is Priya and I', start=0.8, duration=4.88
{
  "summary": "This video provides a comprehensive guide to JavaScript data types, dividing them into primitive types (stored
  "detailed_explanation": "In this tutorial, Priya breaks down JavaScript data types into two main categories: primitive and
  "key_points": [
    "JavaScript data types are divided into primitive and reference types.",
    "Primitive types are stored directly in stack memory, whereas reference types are stored in heap memory with their addre
    "The primitive types covered include Number, String, Boolean, Null, and Undefined.",
    "Due to an old JavaScript glitch, 'typeof null' returns 'object' instead of 'null'.",
    "Reference types include data structures like arrays ([]) and JavaScript objects ({}).",
    "A hands-on counter application is built to demonstrate DOM manipulation, event handling, type conversion using parseInt
  ],
  "insights": [
    "JavaScript is dynamically typed; it internally determines data types at runtime based on the assigned value.",
    "The assignment operator is right-to-left associative, meaning the right side of the expression executes first before be
    "Understanding the distinction between Stack and Heap memory allocation is crucial for mastering performance and referer
    "Using parseInt() is essential when retrieving numbers from DOM text content, as web values are frequently returned as s
    "The historical glitch of 'typeof null === \"object\"' remains unpatched because fixing it would break countless legacy
  ],
  "qna": [
    {
      "question": "What is the main difference in how primitive and reference types are stored in memory?",
      "answer": "Primitive types are stored directly in the stack segment of RAM, while reference types (objects and arrays)
    },
    {
      "question": "Why does 'typeof null' return 'object' in JavaScript?",
      "answer": "This is due to a historical mistake in the early design of JavaScript that has not been corrected to avoid
    },
    {
      "question": "What is the purpose of the parseInt() function in JavaScript?",
      "answer": "It parses a string representation of an integer and converts it into an actual numeric integer type, which
    },
    {
      "question": "How does the assignment operator (=) execute in terms of associativity?",
      "answer": "The assignment operator is right-to-left associative, meaning the right side of the expression is evaluate
    },
    {
      "question": "How was conditional logic used to dynamically style the counter application?",
      "answer": "An 'if...else if...else' construct was used to check the counter value: if it was greater than zero, the el
    }
  ],
  "topics": [
    "javascript",
    "data types",
    "dom manipulation",
    "programming fundamentals",
    "web development",
    "conditional statements"
  ]
}
```

```
def ask_question(transcript, question):
    prompt = f"""
    Answer the question based on this video transcript:
```

```
Transcript:
{transcript}
```

```
Question:
```



```

{question}
"""

response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=prompt
)

return response.text
# Example
print(ask_question(transcript, "What is the main topic?"))

```

Based on the transcript, the main topic of the session is **data types in JavaScript**, specifically focusing on the two main

- Primitive Types:** The basic data types directly supported by the language and stored directly in memory (e.g., numbers).
- Reference Types:** Complex structures stored in the memory's heap segment where variables hold a reference or address to the actual data.

Additionally, the session covers **conditional control constructs (if-else statements)** and demonstrates these concepts with

```

from IPython.display import HTML, display

video_id = "nLj-DGIC_xM" # replace with your video ID
transcript = get_transcript(video_id)
raw_result = analyze_video(transcript)

# Strip markdown fences if Gemini adds them anyway
cleaned = re.sub(r"```json|```$", "", raw_result.strip(), flags=re.MULTILINE).strip()
data = json.loads(cleaned)

summary = data.get("summary", "")
detailed = data.get("detailed_explanation", "")
key_points = data.get("key_points", [])
insights = data.get("insights", [])
qna = data.get("qna", [])
topics = data.get("topics", [])

topics_html = "".join(
    f'<span style="background:#e0e7ff; color:#4338ca; padding:5px 12px; border-radius:14px; '
    f'font-size:13px; font-weight:500; margin:4px; display:inline-block;">{t}</span>'
    for t in topics
)
key_points_html = "".join(
    f'<li style="margin-bottom:8px;"><span style="color:#4338ca;">></span> {k}</li>' for k in key_points
)
insights_html = "".join(
    f'<li style="margin-bottom:8px;"><span style="color:#f59e0b;">💡</span> {i}</li>' for i in insights
)
qna_html = "".join(
    f'<div style="margin-bottom:14px;">
        <div style="font-weight:600; color:#111827; font-size:13px;">Q: {qa.get("question","")}</div>
        <div style="color:#374151; font-size:13px; margin-top:4px;">A: {qa.get("answer","")}</div>
    </div>'
    for qa in qna
)

dashboard = f"""
<div style="font-family:'Segoe UI', Arial, sans-serif; max-width:900px; margin:auto;
    border-radius:14px; overflow:hidden; box-shadow:0 6px 20px rgba(0,0,0,0.12);">

    <div style="background:#312e81; color:white; text-align:center; padding:16px; font-size:18px; font-weight:600;">
        📺 AI YouTube Video Summarizer – Dashboard
    </div>

    <div style="background:#f8f9fc; padding:20px; display:grid; grid-template-columns:1fr 1fr; gap:16px;">

        <!-- Summary -->
        <div style="background:white; border-radius:12px; padding:18px; box-shadow:0 1px 4px rgba(0,0,0,0.06); grid-column:1 / -
            <div style="font-weight:600; color:#111827; margin-bottom:8px;">📌 Summary</div>
            <div style="font-size:14px; color:#374151; line-height:1.6;">{summary}</div>
        </div>

        <!-- Topics -->
        <div style="background:white; border-radius:12px; padding:18px; box-shadow:0 1px 4px rgba(0,0,0,0.06); grid-column:1 / -
            <div style="font-weight:600; color:#111827; margin-bottom:10px;">📌 Topics Covered</div>
            <div>{topics_html}</div>
        </div>

        <!-- Detailed Explanation -->
        <div style="background:white; border-radius:12px; padding:18px; box-shadow:0 1px 4px rgba(0,0,0,0.06); grid-column:1 / -
            <div style="font-weight:600; color:#111827; margin-bottom:8px;">📖 Detailed Explanation</div>

```



```
<div style="font-size:13px; color:#374151; line-height:1.6;">{detailed}</div>
</div>

<!-- Key Points -->
<div style="background:white; border-radius:12px; padding:18px; box-shadow:0 1px 4px rgba(0,0,0,0.06);">
  <div style="font-weight:600; color:#111827; margin-bottom:10px;">🔑 Key Points</div>
  <ul style="list-style:none; padding:0; margin:0; font-size:13px; color:#374151;">{key_points_html}</ul>
</div>

<!-- Insights -->
<div style="background:white; border-radius:12px; padding:18px; box-shadow:0 1px 4px rgba(0,0,0,0.06);">
  <div style="font-weight:600; color:#111827; margin-bottom:10px;">💡 Important Insights</div>
  <ul style="list-style:none; padding:0; margin:0; font-size:13px; color:#374151;">{insights_html}</ul>
</div>

<!-- Q&A -->
<div style="background:white; border-radius:12px; padding:18px; box-shadow:0 1px 4px rgba(0,0,0,0.06); grid-column:1 / 
  <div style="font-weight:600; color:#111827; margin-bottom:12px;">❓ Questions & Answers</div>
  {qna_html}
</div>
</div>
"""

display(HTML(dashboard))
```



```
FetchTranscript(snippets=[FetchTranscriptSnippet(text='Hello everyone, my name is Priya and I', start=0.8, duration=4.88
```

AI YouTube Video Summarizer — Dashboard

Summary

This video tutorial by Priya explains the core concepts of JavaScript data types, categorizing them into primitive and reference types, and explores conditional statements ('if-else'). To consolidate these concepts, she demonstrates a practical hands-on project by building a dynamic Counter web application that changes color based on the current count.

Topics Covered

[JavaScript](#)[Data Types](#)[DOM Manipulation](#)[Conditional Statements](#)[Memory Management](#)[Web Development](#)

Detailed Explanation

The video begins with an in-depth breakdown of primitive data types (Number, String, Boolean, Null, and Undefined), explaining that they are directly supported by the language and stored directly in the stack segment of RAM. The instructor addresses a well-known JavaScript quirk where the `typeof null` returns 'object'. She then introduces reference types (Arrays and Objects), illustrating how they are stored in the heap segment of RAM while variables hold their memory addresses (references) on the stack. After demonstrating these data types in VS Code, she introduces conditional control constructs ('if', 'else if', and 'else'). Finally, she walks through building a functional Counter web application, showing how to link JavaScript to HTML, parse strings to numbers with `parseInt()`, handle user click events for incrementing, decrementing, and resetting the counter, and apply dynamic inline styles based on conditional logic.

Key Points

- ▶ JavaScript data types are broadly classified into primitive and reference types.
- ▶ Primitive types (Number, String, Boolean, Null, Undefined) are stored directly in the stack memory segment.
- ▶ Reference types (such as Arrays and Objects) are stored in heap

Important Insights

- 💡 JavaScript does not differentiate between integers and floating-point numbers; both fall under the single 'Number' primitive type.
- 💡 Variables holding reference types do not contain the actual data structure but rather a memory reference, meaning copying them copies the reference, not the value itself.

